

LogPulse AI - Incident Report

Generated : 2026-06-01 21:37:52 UTC

Trace ID : 647c96d4-e63d-4712-864c-2f6433041b46

1 - Incident Summary

Incident ID : 647c96d4
Category : Unknown
Severity : SeverityLevel.INFO
Source : Fallback
Cache : MISS

2 - Raw Log

2026-01-15T22:14:07Z FATAL kubelet: Pod worker-api-7d9f8b-xk2q OOMKilled. Container exceeded memory limit of 256Mi. Exit code 137. Restart count: 14. Node: worker-1. Namespace: production.

3 - Classification

Category : Unknown
Severity : SeverityLevel.INFO
Summary : Fallback classification triggered due to inference failure.
Source : Fallback

4 - Diagnosis

Root Cause : The 'worker-api' container is consistently exceeding its configured memory limit of 256Mi, leading to repeated OOMKills and restarts.
Confidence : 95%
Reasoning : The raw log explicitly states 'OOMKilled. Container exceeded memory limit of 256Mi' with 14 restarts, and the retrieved playbook directly addresses 'Pods OOMKilled due to insufficient memory', indicating the memory limit is set too low for the application's needs.
Source Tier : llm
Used History : No
Used Playbook : Yes

5 - Proposed Solution

Explanation:

Gemini unavailable; remediation derived from playbook 'Pods OOMKilled due to insufficient memory' (RAG confidence=0.7875). Review the playbook steps and adapt commands to the live environment before execution.

Steps:

- 1. Identify the specific pods and deployments experiencing OOMKills using `kubectl get events` and `kubectl describe pod <pod-name>`.
- 2. Check the application logs within the affected pods for memory-related errors.
- 3. Review the `memory.limit` and `memory.request` defined in the pod's container spec.
- 4. If the application is known to have increased memory requirements, increment the `memory.limit` for the affected deployment in its YAML.
- 5. If a memory leak is suspected, collaborate with the development team to analyze heap dumps or application-specific memory metrics.
- 6. Consider scaling out the deployment or the node pool if node-wide memory pressure is observed consistently.
- 7. name: my-container

Commands:

```
$ kubectl apply -f deployment.yaml
$ kubectl rollout status deployment/my-service -w
```

6 - Risk Assessment

Risk Level : RISKLEVEL.SAFE

Safe Commands (2):

- [OK] `kubectl apply -f deployment.yaml`
- [OK] `kubectl rollout status deployment/my-service -w`

Flagged Commands (0):

(none)

7 - Operator Decision

Decision : OPERATORDECISION.APPROVED
Executed At : 2026-06-01T18:37:48.311557+00:00

Sandbox Results:

Command : `kubectl apply -f deployment.yaml`
Exit : 0
Time : 0ms

Output:

```
[simulation] applied deployment.yaml (no manifest state in simulator)
```

Command : `kubectl rollout status deployment/my-service -w`
Exit : 1
Time : 0ms

8 - Historical Context

Times Seen : 1

First occurrence -- no prior incidents.

9 - Incident Timeline

```
[18:32:18] INCIDENT_RECEIVED -> Incident received (trace_id=647c96d4...)
[18:32:18] CACHE_MISS -> Cache miss -- routing to classifier
[18:32:21] INCIDENT_CLASSIFIED -> Fallback / Info
[18:32:27] RAG_RETRIEVED -> Playbook retrieved (fallback) conf=0.79
[18:32:32] ROOT_CAUSE_GENERATED -> The 'worker-api' container is consistently exceeding its
configured memory limit of 256Mi, leading to repeated OOMKills
[18:34:21] SOLUTION_GENERATED -> 7 step(s) from playbook
[18:34:21] RISK_ASSESSED -> safe -- 0 flagged
[18:37:48] SANDBOX_EXECUTED -> kubectl apply -f deployment.yaml (exit 0)
[18:37:48] SANDBOX_FAILED -> kubectl rollout status deployment/my-service -w (exit 1)
[18:37:48] WORKFLOW_COMPLETED -> Completed in 329572ms
```